

[--- MARKDOWN CELL ---]

All Algorithms From Scratch - Complete Mathematical Implementation

This notebook implements all 5 ML algorithms completely from scratch, including the core mathematical components:

1. **KNN** - Euclidean distance, majority voting
2. **Linear Regression** - MSE loss, gradient descent
3. **Logistic Regression** - Sigmoid, binary cross-entropy, gradient descent
4. **Naive Bayes** - Prior, Likelihood (Gaussian PDF), Posterior calculation
5. **Decision Tree** - Gini impurity, Information Gain, recursive tree building

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score, confusion_matrix
np.random.seed(42)
```

[--- MARKDOWN CELL ---]

1. K-Nearest Neighbors (KNN) From Scratch

Mathematical Foundation:

- **Euclidean Distance:** $d(p,q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$
- **Manhattan Distance:** $d(p,q) = \sum_{i=1}^n |p_i - q_i|$
- **Prediction:** Majority vote of k nearest neighbors

[--- CODE CELL ---]

```
class KNNFromScratch:
    """K-Nearest Neighbors with distance calculations from scratch."""
    def __init__(self, k=3, distance_metric='euclidean'):
        self.k = k
        self.distance_metric = distance_metric
        self.X_train = None
        self.y_train = None
    def euclidean_distance(self, x1, x2):
        """Euclidean distance: sqrt(sum((x1 - x2)^2))"""
        return np.sqrt(np.sum((x1 - x2) ** 2))
    def manhattan_distance(self, x1, x2):
        """Manhattan distance: sum(|x1 - x2|)"""
        return np.sum(np.abs(x1 - x2))
    def compute_distance(self, x1, x2):
        """Compute distance based on selected metric."""
        if self.distance_metric == 'euclidean':
            return self.euclidean_distance(x1, x2)
        elif self.distance_metric == 'manhattan':
            return self.manhattan_distance(x1, x2)
    def fit(self, X, y):
        """Store training data (lazy learning)."""
        self.X_train = np.asarray(X)
        self.y_train = np.asarray(y)
        return self
```

```

def predict_single(self, x):
    """Predict class for a single sample using majority vote."""
    # Compute distances to all training samples
    distances = [self.compute_distance(x, x_train) for x_train in self.X_train]
    # Get k nearest neighbors
    k_indices = np.argsort(distances)[:self.k]
    k_labels = self.y_train[k_indices]
    # Majority vote (count occurrences)
    unique_labels, counts = np.unique(k_labels, return_counts=True)
    return unique_labels[np.argmax(counts)]

def predict(self, X):
    return np.array([self.predict_single(x) for x in X])

def score(self, X, y):
    return np.mean(self.predict(X) == y)

```

[--- CODE CELL ---]

```

# Test KNN
df_bc = pd.read_csv('../datasets/breast_cancer.csv')
X, y = df_bc.drop('target', axis=1).values, df_bc['target'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_s, X_test_s = scaler.fit_transform(X_train), scaler.transform(X_test)
knn = KNNFromScratch(k=5)
knn.fit(X_train_s, y_train)
print(f"KNN Accuracy: {knn.score(X_test_s, y_test):.4f}")

```

[--- MARKDOWN CELL ---]

2. Linear Regression From Scratch

****Mathematical Foundation:****

- ****Hypothesis:**** $h(x) = w \cdot x + b$

- ****MSE Loss:**** $J(w, b) = \frac{1}{2m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$

- ****Gradient:**** $\frac{\partial J}{\partial w} = \frac{1}{m} \sum (h(x) - y) \cdot x$, $\frac{\partial J}{\partial b} = \frac{1}{m} \sum (h(x) - y)$

[--- CODE CELL ---]

```

class LinearRegressionFromScratch:
    """Linear Regression with gradient descent from scratch."""

    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.w = None
        self.b = None
        self.cost_history = []

    def mse_loss(self, y_true, y_pred):
        """Mean Squared Error: (1/2m) * sum((y_pred - y_true)^2)"""
        m = len(y_true)
        return (1 / (2 * m)) * np.sum((y_pred - y_true) ** 2)

    def compute_gradients(self, X, y, y_pred):
        """Compute gradients for weights and bias."""
        m = len(y)
        error = y_pred - y
        dw = (1 / m) * np.dot(X.T, error) # Gradient w.r.t. weights
        db = (1 / m) * np.sum(error)      # Gradient w.r.t. bias
        return dw, db

    def fit(self, X, y):
        m, n = X.shape

```

```

self.w = np.zeros(n)
self.b = 0
self.cost_history = []
for _ in range(self.epochs):
    # Forward pass:  $h(x) = w \cdot x + b$ 
    y_pred = np.dot(X, self.w) + self.b
    # Compute cost
    cost = self.mse_loss(y, y_pred)
    self.cost_history.append(cost)
    # Compute gradients
    dw, db = self.compute_gradients(X, y, y_pred)
    # Gradient descent update
    self.w -= self.lr * dw
    self.b -= self.lr * db
return self

def predict(self, X):
    return np.dot(X, self.w) + self.b

def r2_score(self, X, y):
    """ $R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$ """
    y_pred = self.predict(X)
    ss_res = np.sum((y - y_pred) ** 2)
    ss_tot = np.sum((y - np.mean(y)) ** 2)
    return 1 - (ss_res / ss_tot)

```

[--- CODE CELL ---]

```

# Test Linear Regression
df_housing = pd.read_csv('../datasets/california_housing.csv')
X, y = df_housing.drop('target', axis=1).values, df_housing['target'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_s, X_test_s = scaler.fit_transform(X_train), scaler.transform(X_test)
lr = LinearRegressionFromScratch(lr=0.1, epochs=1000)
lr.fit(X_train_s, y_train)
print(f"Linear Regression  $R^2$ : {lr.r2_score(X_test_s, y_test):.4f}")

```

[--- MARKDOWN CELL ---]

3. Logistic Regression From Scratch

****Mathematical Foundation:****

- ****Sigmoid:**** $\sigma(z) = \frac{1}{1 + e^{-z}}$
- ****Binary Cross-Entropy:**** $J = -\frac{1}{m} \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$
- ****Gradient:**** $\frac{\partial J}{\partial w} = \frac{1}{m} X^T (\sigma(z) - y)$

[--- CODE CELL ---]

```

class LogisticRegressionFromScratch:
    """Logistic Regression with sigmoid and cross-entropy from scratch."""
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.w = None
        self.b = None
        self.cost_history = []
    def sigmoid(self, z):
        """Sigmoid function:  $1 / (1 + \exp(-z))$ """
        z = np.clip(z, -500, 500) # Prevent overflow
        return 1 / (1 + np.exp(-z))

```

```

def binary_cross_entropy(self, y_true, y_pred):
    """BCE = -1/m * sum(y*log(p) + (1-y)*log(1-p))"""
    m = len(y_true)
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return (-1 / m) * np.sum(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

def fit(self, X, y):
    m, n = X.shape
    self.w = np.zeros(n)
    self.b = 0
    self.cost_history = []
    for _ in range(self.epochs):
        # Forward pass
        z = np.dot(X, self.w) + self.b
        y_pred = self.sigmoid(z)
        # Compute cost
        cost = self.binary_cross_entropy(y, y_pred)
        self.cost_history.append(cost)
        # Compute gradients
        error = y_pred - y
        dw = (1 / m) * np.dot(X.T, error)
        db = (1 / m) * np.sum(error)
        # Update
        self.w -= self.lr * dw
        self.b -= self.lr * db
    return self

def predict_proba(self, X):
    return self.sigmoid(np.dot(X, self.w) + self.b)

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X) >= threshold).astype(int)

def score(self, X, y):
    return np.mean(self.predict(X) == y)

```

[--- CODE CELL ---]

```

# Test Logistic Regression
df_bc = pd.read_csv('../datasets/breast_cancer.csv')
X, y = df_bc.drop('target', axis=1).values, df_bc['target'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_s, X_test_s = scaler.fit_transform(X_train), scaler.transform(X_test)
logreg = LogisticRegressionFromScratch(lr=0.1, epochs=1000)
logreg.fit(X_train_s, y_train)
print(f"Logistic Regression Accuracy: {logreg.score(X_test_s, y_test):.4f}")

```

[--- MARKDOWN CELL ---]

4. Naive Bayes From Scratch

****Mathematical Foundation:****

Prior Probability:

$$P(y) = \frac{\text{Count of class } y}{\text{Total samples}}$$

Likelihood $P(X|y)$:

****For Continuous (Gaussian):****

$$P(X|Y) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(X-\mu)^2}{2\sigma^2}\right)$$

****For Categorical:****

$$P(X=v|Y=c) = \frac{\text{Count of } X=v \text{ in class } c}{\text{Total count of class } c}$$

Posterior (Bayes' Theorem):

$$P(Y|X) \propto P(Y) \cdot \prod_{i=1}^n P(X_i|Y)$$

[--- CODE CELL ---]

```
class GaussianNaiveBayesFromScratch:
```

```
    """Gaussian Naive Bayes with complete probability calculation from scratch."""
```

```
    def __init__(self):
```

```
        self.classes = None
```

```
        self.priors = None      # P(y) for each class
```

```
        self.means = None      # Mean of each feature per class
```

```
        self.variances = None  # Variance of each feature per class
```

```
    def compute_prior(self, y):
```

```
        """Compute prior probability P(y) = count(y) / total"""
```

```
        self.classes, counts = np.unique(y, return_counts=True)
```

```
        self.priors = counts / len(y)
```

```
        print(f"Prior probabilities P(y):")
```

```
        for cls, prior in zip(self.classes, self.priors):
```

```
            print(f" P(y={cls}) = {counts[list(self.classes).index(cls)]/len(y)} = {prior:.4f}")
```

```
    def compute_class_statistics(self, X, y):
```

```
        """Compute mean (?) and variance (?) for each feature per class."""
```

```
        n_classes = len(self.classes)
```

```
        n_features = X.shape[1]
```

```
        self.means = np.zeros((n_classes, n_features))
```

```
        self.variances = np.zeros((n_classes, n_features))
```

```
        print(f"\nClass statistics (?, ?) for each class:")
```

```
        for i, cls in enumerate(self.classes):
```

```
            X_class = X[y == cls]
```

```
            self.means[i] = X_class.mean(axis=0)
```

```
            self.variances[i] = X_class.var(axis=0) + 1e-9 # Add small value for stability
```

```
            print(f" Class {cls}: ? = {self.means[i][:3]}..., ? = {self.variances[i][:3]}...")
```

```
    def gaussian_pdf(self, x, mean, var):
```

```
        """Gaussian PDF: P(x|y) = 1/sqrt(2???) * exp(-(x-?)^2/(2??))"""
```

```
        coefficient = 1 / np.sqrt(2 * np.pi * var)
```

```
        exponent = np.exp(-((x - mean) ** 2) / (2 * var))
```

```
        return coefficient * exponent
```

```
    def compute_likelihood(self, x, class_idx):
```

```
        """Compute P(X|y) = product of P(xi|y) for all features."""
```

```
        mean = self.means[class_idx]
```

```
        var = self.variances[class_idx]
```

```
        # Compute Gaussian PDF for each feature
```

```
        likelihoods = self.gaussian_pdf(x, mean, var)
```

```
        # Product of all feature likelihoods (using log to avoid underflow)
```

```
        return np.prod(likelihoods)
```

```
    def compute_log_likelihood(self, x, class_idx):
```

```
        """Compute log P(X|y) using log of Gaussian for numerical stability."""
```

```
        mean = self.means[class_idx]
```

```
        var = self.variances[class_idx]
```

```
        # Log Gaussian: -0.5*log(2???) - (x-?)^2/(2??)
```

```
        log_coeff = -0.5 * np.log(2 * np.pi * var)
```

```
        log_exp = -((x - mean) ** 2) / (2 * var)
```

```
        return np.sum(log_coeff + log_exp)
```

```
    def fit(self, X, y):
```

```
        """Fit the model by computing priors and class statistics."""
```

```
        X, y = np.asarray(X), np.asarray(y)
```

```
        # Step 1: Compute Prior P(y)
```

```
        self.compute_prior(y)
```

```

# Step 2: Compute Mean and Variance for each class
self.compute_class_statistics(X, y)
return self

def predict_single(self, x):
    """Predict class for a single sample using Bayes' theorem."""
    posteriors = []
    for i, cls in enumerate(self.classes):
        #  $P(y|X) \propto P(y) * P(X|y)$ 
        log_prior = np.log(self.priors[i])
        log_likelihood = self.compute_log_likelihood(x, i)
        log_posterior = log_prior + log_likelihood
        posteriors.append(log_posterior)

    # Return class with highest posterior
    return self.classes[np.argmax(posteriors)]

def predict(self, X):
    return np.array([self.predict_single(x) for x in X])

def score(self, X, y):
    return np.mean(self.predict(X) == y)

def predict_with_details(self, x):
    """Predict with detailed probability breakdown."""
    print(f"\nPredicting for sample: {x[:3]}...")
    print("="*60)
    posteriors = []
    for i, cls in enumerate(self.classes):
        prior = self.priors[i]
        log_prior = np.log(prior)
        log_likelihood = self.compute_log_likelihood(x, i)
        log_posterior = log_prior + log_likelihood
        print(f"\nClass {cls}:")
        print(f"  P(y={cls}) = {prior:.4f}")
        print(f"  log P(X|y={cls}) = {log_likelihood:.4f}")
        print(f"  log P(y={cls}|X)  $\propto$  {log_posterior:.4f}")
        posteriors.append(log_posterior)

    # Normalize posteriors
    max_log = max(posteriors)
    exp_posteriors = np.exp(np.array(posteriors) - max_log)
    normalized = exp_posteriors / np.sum(exp_posteriors)
    print(f"\nNormalized Posteriors:")
    for i, cls in enumerate(self.classes):
        print(f"  P(y={cls}|X) = {normalized[i]:.4f}")
    prediction = self.classes[np.argmax(posteriors)]
    print(f"\nPrediction: {prediction}")
    return prediction

```

[--- CODE CELL ---]

```

# Test Naive Bayes
df_bc = pd.read_csv('../datasets/breast_cancer.csv')
X, y = df_bc.drop('target', axis=1).values, df_bc['target'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
nb = GaussianNaiveBayesFromScratch()
nb.fit(X_train, y_train)
print(f"\nNaive Bayes Accuracy: {nb.score(X_test, y_test):.4f}")

```

[--- CODE CELL ---]

```

# Show detailed prediction for one sample
nb.predict_with_details(X_test[0])

```

[--- MARKDOWN CELL ---]

5. Decision Tree From Scratch

Mathematical Foundation:

Gini Impurity:

$$Gini(D) = 1 - \sum_{i=1}^{|C|} p_i^2$$

Entropy:

$$Entropy(D) = -\sum_{i=1}^{|C|} p_i \log_2(p_i)$$

Information Gain:

$$IG(D, A) = Impurity(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} \cdot Impurity(D_v)$$

[--- CODE CELL ---]

```

class DecisionTreeFromScratch:
    """Decision Tree with Gini/Entropy and Information Gain from scratch."""
    def __init__(self, max_depth=None, min_samples_split=2, criterion='gini'):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.criterion = criterion
        self.tree = None

    def gini_impurity(self, y):
        """Gini = 1 - sum(p_i^2)"""
        if len(y) == 0:
            return 0
        _, counts = np.unique(y, return_counts=True)
        probabilities = counts / len(y)
        return 1 - np.sum(probabilities ** 2)

    def entropy(self, y):
        """Entropy = -sum(p_i * log2(p_i))"""
        if len(y) == 0:
            return 0
        _, counts = np.unique(y, return_counts=True)
        probabilities = counts / len(y)
        # Avoid log(0)
        probabilities = probabilities[probabilities > 0]
        return -np.sum(probabilities * np.log2(probabilities))

    def impurity(self, y):
        """Calculate impurity based on criterion."""
        if self.criterion == 'gini':
            return self.gini_impurity(y)
        else:
            return self.entropy(y)

    def information_gain(self, y, y_left, y_right):
        """IG = Impurity(parent) - weighted_avg(Impurity(children))"""
        n = len(y)
        n_left, n_right = len(y_left), len(y_right)
        if n_left == 0 or n_right == 0:
            return 0
        parent_impurity = self.impurity(y)
        weighted_child_impurity = (n_left / n) * self.impurity(y_left) + \
            (n_right / n) * self.impurity(y_right)
        return parent_impurity - weighted_child_impurity

    def find_best_split(self, X, y):
        """Find the best feature and threshold to split on."""
        best_gain = -float('inf')
        best_feature = None

```

```

best_threshold = None
n_features = X.shape[1]
for feature_idx in range(n_features):
    thresholds = np.unique(X[:, feature_idx])
    for threshold in thresholds:
        # Split data
        left_mask = X[:, feature_idx] <= threshold
        right_mask = ~left_mask
        if np.sum(left_mask) == 0 or np.sum(right_mask) == 0:
            continue
        # Calculate information gain
        gain = self.information_gain(y, y[left_mask], y[right_mask])
        if gain > best_gain:
            best_gain = gain
            best_feature = feature_idx
            best_threshold = threshold
    return best_feature, best_threshold, best_gain

def build_tree(self, X, y, depth=0):
    """Recursively build the decision tree."""
    n_samples, n_features = X.shape
    n_classes = len(np.unique(y))
    # Stopping conditions
    if (self.max_depth is not None and depth >= self.max_depth) or \
        n_classes == 1 or \
        n_samples < self.min_samples_split:
        # Return leaf node with majority class
        unique, counts = np.unique(y, return_counts=True)
        return {'leaf': True, 'class': unique[np.argmax(counts)]}
    # Find best split
    best_feature, best_threshold, best_gain = self.find_best_split(X, y)
    if best_feature is None:
        unique, counts = np.unique(y, return_counts=True)
        return {'leaf': True, 'class': unique[np.argmax(counts)]}
    # Split data
    left_mask = X[:, best_feature] <= best_threshold
    right_mask = ~left_mask
    # Recursively build subtrees
    left_subtree = self.build_tree(X[left_mask], y[left_mask], depth + 1)
    right_subtree = self.build_tree(X[right_mask], y[right_mask], depth + 1)
    return {
        'leaf': False,
        'feature': best_feature,
        'threshold': best_threshold,
        'gain': best_gain,
        'left': left_subtree,
        'right': right_subtree
    }

def fit(self, X, y):
    self.tree = self.build_tree(np.asarray(X), np.asarray(y))
    return self

def predict_single(self, x, node=None):
    if node is None:
        node = self.tree
    if node['leaf']:
        return node['class']
    if x[node['feature']] <= node['threshold']:
        return self.predict_single(x, node['left'])
    else:
        return self.predict_single(x, node['right'])

def predict(self, X):

```



```

    return np.array([self.predict_single(x) for x in X])

def score(self, X, y):
    return np.mean(self.predict(X) == y)

def print_tree(self, node=None, indent=""):
    """Print the decision tree structure."""
    if node is None:
        node = self.tree

    if node['leaf']:
        print(f"{indent}Leaf: Class = {node['class']}")
    else:
        print(f"{indent}Feature[{node['feature']}] <= {node['threshold']:.4f} (Gain: {node['gain']:.4f})")
        print(f"{indent}??? Left:")
        self.print_tree(node['left'], indent + "?  ")
        print(f"{indent}??? Right:")
        self.print_tree(node['right'], indent + "  ")

```

[--- CODE CELL ---]

```

# Test Decision Tree
df_iris = pd.read_csv('../datasets/iris.csv')
X, y = df_iris.drop('target', axis=1).values, df_iris['target'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
dt = DecisionTreeFromScratch(max_depth=3, criterion='gini')
dt.fit(X_train, y_train)
print(f"Decision Tree Accuracy: {dt.score(X_test, y_test):.4f}")

```

[--- CODE CELL ---]

```

# Print tree structure
print("\nDecision Tree Structure:")
print("="*60)
dt.print_tree()

```

[--- CODE CELL ---]

```

# Demonstrate Gini and Entropy calculation
print("Gini Impurity Demonstration:")
print("="*60)

# Pure node (all same class)
pure = np.array([1, 1, 1, 1, 1])
print(f"Pure node [1,1,1,1,1]: Gini = {dt.gini_impurity(pure):.4f}")

# Balanced split
balanced = np.array([0, 0, 1, 1])
print(f"Balanced [0,0,1,1]: Gini = {dt.gini_impurity(balanced):.4f}")

# Imbalanced
imbalanced = np.array([0, 0, 0, 1])
print(f"Imbalanced [0,0,0,1]: Gini = {dt.gini_impurity(imbalanced):.4f}")

print("\nEntropy Demonstration:")
print(f"Pure node: Entropy = {dt.entropy(pure):.4f}")
print(f"Balanced: Entropy = {dt.entropy(balanced):.4f}")
print(f"Imbalanced: Entropy = {dt.entropy(imbalanced):.4f}")

```

[--- MARKDOWN CELL ---]

```

---
## Summary: All Algorithms Comparison

```

[--- CODE CELL ---]

```

# Compare all algorithms on Breast Cancer dataset
print("All Algorithms Comparison on Breast Cancer Dataset")
print("="*60)

```

```
df_bc = pd.read_csv('../datasets/breast_cancer.csv')
X, y = df_bc.drop('target', axis=1).values, df_bc['target'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_s, X_test_s = scaler.fit_transform(X_train), scaler.transform(X_test)
results = []
```

[--- CODE CELL ---]

```
# Visualization
names = [r[0] for r in results]
accs = [r[1] for r in results]
plt.figure(figsize=(12, 6))
colors = plt.cm.viridis(np.linspace(0.2, 0.8, len(names)))
bars = plt.bar(names, accs, color=colors, edgecolor='black')
for bar, acc in zip(bars, accs):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
             f'{acc:.4f}', ha='center', va='bottom', fontsize=11)
plt.ylabel('Accuracy', fontsize=12)
plt.title('All Algorithms From Scratch - Accuracy Comparison', fontsize=14)
plt.ylim(0.85, 1.0)
plt.grid(True, alpha=0.3, axis='y')
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```